VendorBridge ERP

Complete Backend Architecture & Development Specification

Procurement & Vendor Management ERP — Backend Engineering Document

Document Type	Backend Engineering Specification
Project	VendorBridge — Procurement & Vendor Management ERP
Version	v1.0 — Full System Documentation
Tech Stack	Node.js / Express + PostgreSQL + Redis + JWT
Audience	Backend Engineers, DevOps, Technical Leads
Modules	11 Screens — Auth, Dashboard, Vendors, RFQ, Quotations, Approvals, PO/Invoices, Activity

Scope of this Document: This document provides exhaustive backend implementation guidance for every module of the VendorBridge ERP. It covers technology stack, project structure, database schema, API endpoint design, business logic, middleware, authentication, error handling, file operations, email services, PDF generation, real-time features, and deployment — everything required to build a production-grade backend that powers the 11-screen frontend.

Table of Contents

PART 7	**Vendor Quotation Submission (Screen 5)**
7.1	Quotation API
7.2	Quotation Versioning
7.3	Submission Validation Rules
PART 8	**Quotation Comparison Engine (Screen 6)**
8.1	Side-by-Side Comparison API
8.2	Lowest-Price Detection Logic
8.3	Scoring Algorithm
PART 9	**Approval Workflow Engine (Screen 7)**
9.1	Approval State Machine
9.2	Approve / Reject API
9.3	Multi-Level Approval Chain
9.4	Notification Dispatch on Status Change
PART 10	**Purchase Orders & Invoice Generation (Screen 8)**
10.1	Auto-PO Number Generation
10.2	PO & Invoice CRUD API
10.3	Tax Calculation Service
10.4	PDF Generation (PDFKit / Puppeteer)
10.5	Print & Email Dispatch
PART 11	**Activity Logs & Notifications (Screen 9)**
11.1	Audit Log Schema & Service
11.2	Real-Time Notifications (Socket.io)
11.3	Notification Preferences
PART 12	**Reports & Analytics (Screen 10)**
12.1	Analytics Aggregation Queries
12.2	Vendor Performance Scoring
12.3	Exportable Reports (PDF/XLSX/CSV)
12.4	Date-Range Filtering
PART 13	**User Management & Admin (Screen 11)**
13.1	Admin User CRUD API
13.2	Role Assignment & Suspension
13.3	Invite User Flow (Email Token)
13.4	Login Activity Tracking

PART 1 — Technology Stack & Project Architecture

Foundation layer: runtime, framework, database, tooling, and folder structure

1.1 Recommended Technology Stack

VendorBridge backend is built on a modern Node.js monolith with a clear separation of concerns. The following table documents every technology choice with the specific version pinned and the rationale for selection.

Layer	Technology	Version	Purpose / Rationale
Runtime	Node.js	20 LTS	Long-term support, native ESM, excellent async I/O for ERP
Framework	Express.js	4.18+	Minimal, unopinionated; full control over middleware stack
Language	TypeScript	5.x	Type safety across models, controllers, and services — critic
ORM	Prisma	5.x	Type-safe DB access, migration support, schema-first design
Database	PostgreSQL	15+	ACID compliance, JSON support, window functions for analy
Cache	Redis	7+	Session caching, rate limiting, pub/sub for notifications
Auth	JWT + bcrypt	jsonwebtoken 9 / bcrypt 5	Stateless auth with secure password hashing
File Upload	Multer + AWS S3	1.4 / 3.x SDK	Multipart form handling; S3 for production file storage
PDF Gen	PDFKit / Puppeteer	0.13 / 21+	PDFKit for programmatic invoices; Puppeteer for HTML-to-PD
Email	Nodemailer + SendGrid	6.9 / 7.x	SMTP-based dev; SendGrid for production deliverability
Validation	Zod	3.x	Schema-first validation — integrates cleanly with TypeScript
Real-time	Socket.io	4.6+	WebSocket-based notifications for approval/RFQ events
API Docs	Swagger / OpenAPI	swagger-jsdoc 6	Auto-generated docs; essential for team collaboration
Testing	Jest + Supertest	29 / 6+	Unit + integration test coverage for all API layers
Containerize	Docker + Compose	latest	Consistent dev/prod environments; compose for multi-service
Process Mgr	PM2	5.x	Cluster mode, zero-downtime reload in production
Logger	Winston	3.x	Structured JSON logging with log levels and file transports
CORS/Security	Helmet + cors	7 / 2.8+	Security headers; CORS policy per environment
Rate Limit	express-rate-limit	7.x	Protect auth endpoints from brute-force attacks

1.2 Project Folder Structure

The project follows a feature-first modular architecture. Each feature (vendors, rfq, quotations, approvals, purchase-orders, invoices, reports) lives in its own folder containing routes, controller, service, and validators. Shared utilities (email, pdf, storage) live at the top-level src/utils/.

```
vendorbridge-backend/
■■■ src/
■ ■■■ app.ts # Express app factory (middleware stack)
■ ■■■ server.ts # HTTP server entry point + Socket.io init
■ ■■■ config/
■ ■ ■■■ db.ts # Prisma client singleton
■ ■ ■■■ redis.ts # Redis client singleton
■ ■ ■■■ env.ts # Typed env loader (Zod schema)
■ ■ ■■■ swagger.ts # OpenAPI spec config
■ ■■■ middleware/
■ ■ ■■■ protect.ts # JWT verification → req.user
■ ■ ■■■ roleGuard.ts # Role-based access guard factory
■ ■ ■■■ errorHandler.ts # Global error handler
■ ■ ■■■ validate.ts # Zod request validation wrapper
■ ■ ■■■ rateLimiter.ts # express-rate-limit instances
■ ■ ■■■ upload.ts # Multer config (disk/memory/S3)
■ ■■■ modules/
■ ■ ■■■ auth/
■ ■ ■ ■■■ auth.routes.ts
■ ■ ■ ■■■ auth.controller.ts
■ ■ ■ ■■■ auth.service.ts
■ ■ ■ ■■■ auth.schema.ts
■ ■ ■■■ dashboard/
■ ■ ■■■ vendors/
■ ■ ■■■ rfq/
■ ■ ■■■ quotations/
■ ■ ■■■ approvals/
■ ■ ■■■ purchase-orders/
■ ■ ■■■ invoices/
■ ■ ■■■ activity-logs/
■ ■ ■■■ reports/
■ ■ ■■■ users/
■ ■■■ utils/
■ ■ ■■■ email.util.ts # Nodemailer / SendGrid wrapper
■ ■ ■■■ pdf.util.ts # PDFKit invoice builder
■ ■ ■■■ storage.util.ts # S3 / local file operations
■ ■ ■■■ pagination.util.ts # Cursor + offset pagination
■ ■ ■■■ audit.util.ts # Audit log writer
■ ■ ■■■ gst.util.ts # GST validation & calculation
■ ■■■ events/
■ ■ ■■■ emitter.ts # EventEmitter singleton
■ ■ ■■■ handlers/ # Domain event listeners
■ ■■■ types/
■ ■■■ express.d.ts # Augmented Request (req.user)
■ ■■■ common.ts # Shared TS types
■■■ prisma/
■ ■■■ schema.prisma
■ ■■■ migrations/
■ ■■■ seed.ts
■■■ tests/
■ ■■■ unit/
■ ■■■ integration/
■■■ docker/
■ ■■■ Dockerfile
■ ■■■ docker-compose.yml
■■■ .env.example
■■■ tsconfig.json
■■■ package.json
```

1.3 Environment Configuration (.env)

```
# ■■ Server ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
NODE_ENV=development
PORT=5000
API_PREFIX=/api/v1

# ■■ Database ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
DATABASE_URL=postgresql://user:password@localhost:5432/vendorbridge

# ■■ Redis ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
REDIS_URL=redis://localhost:6379

# ■■ JWT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
JWT_SECRET=super_secret_key_min_32_chars
JWT_EXPIRES_IN=7d
JWT_REFRESH_SECRET=refresh_secret_key
JWT_REFRESH_EXPIRES_IN=30d

# ■■ Email ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
SMTP_HOST=smtp.sendgrid.net
```

```
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASS=SG.xxxxxxxxxxxxxxxxxxx
EMAIL_FROM=noreply@vendorbridge.com

# ■■ AWS S3 ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
AWS_REGION=ap-south-1
AWS_ACCESS_KEY_ID=AKIAxxxxxxxxxx
AWS_SECRET_ACCESS_KEY=xxxxxxxxxxxxxxxx
AWS_S3_BUCKET=vendorbridge-files

# ■■ App ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
FRONTEND_URL=http://localhost:3000
MAX_FILE_SIZE_MB=10
BCRYPT_ROUNDS=12

# ■■ PDF ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium-browser
```

1.4 Core Dependencies (package.json)

```
{
"dependencies": {
"express": "^4.18.2",
"@prisma/client": "^5.7.0",
"bcryptjs": "^2.4.3",
"jsonwebtoken": "^9.0.2",
"zod": "^3.22.4",
"multer": "^1.4.5-lts.1",
"@aws-sdk/client-s3": "^3.454.0",
"nodemailer": "^6.9.7",
"pdfkit": "^0.14.0",
"puppeteer": "^21.5.2",
"socket.io": "^4.6.2",
"redis": "^4.6.11",
"ioredis": "^5.3.2",
"helmet": "^7.1.0",
"cors": "^2.8.5",
"express-rate-limit": "^7.1.5",
"winston": "^3.11.0",
"xlsx": "^0.18.5",
"csv-stringify": "^6.4.4",
"swagger-jsdoc": "^6.2.8",
"swagger-ui-express": "^5.0.0"
},
"devDependencies": {
"typescript": "^5.3.2",
"prisma": "^5.7.0",
"@types/express": "^4.17.21",
"jest": "^29.7.0",
"supertest": "^6.3.3",
"ts-jest": "^29.1.1",
"nodemon": "^3.0.2",
"ts-node": "^10.9.2"
}
}
```

PART 2 — Database Design & Schema

Complete PostgreSQL schema via Prisma — all tables, relations, enums, and indexes

2.1 Entity-Relationship Overview

The VendorBridge data model has 12 primary entities. The following diagram describes their relationships and cardinalities:

Entity	Relates To	Cardinality	Join / FK
users	vendors	1 user → 0..1 vendor profile	vendors.user_id
users	rfqs	1 user → many rfqs (creator)	rfqs.created_by
vendors	rfq_vendors	many-to-many with rfqs	rfq_vendors pivot
rfqs	rfq_items	1 rfq → many line items	rfq_items.rfq_id
rfqs	quotations	1 rfq → many quotations	quotations.rfq_id
quotations	quotation_items	1 quotation → many items	quotation_items.quotation_id
quotations	approvals	1 quotation → 1 approval chain	approvals.quotation_id
approvals	purchase_orders	1 approval → 1 PO	purchase_orders.approval_id
purchase_orders	invoices	1 PO → 1 invoice	invoices.po_id
users	activity_logs	1 user → many logs	activity_logs.user_id
users	notifications	1 user → many notifications	notifications.user_id

2.2 Complete Prisma Schema (prisma/schema.prisma)

Every model below is production-grade with proper indexes, cascades, and audit timestamps.

```
generator client {
provider = "prisma-client-js"
}

datasource db {
provider = "postgresql"
url = env("DATABASE_URL")
}

// ■■■ ENUMS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

enum Role {
PROCUREMENT_OFFICER
VENDOR
MANAGER
ADMIN
}

enum VendorStatus {
PENDING_APPROVAL
ACTIVE
INACTIVE
BLACKLISTED
}

enum RFQStatus {
```

```
DRAFT
PUBLISHED
CLOSED
CANCELLED
}

enum QuotationStatus {
SUBMITTED
UNDER_REVIEW
SHORTLISTED
REJECTED
ACCEPTED
}

enum ApprovalStatus {
PENDING
APPROVED
REJECTED
ESCALATED
}

enum POStatus {
DRAFT
ISSUED
ACKNOWLEDGED
DELIVERED
CANCELLED
}

enum InvoiceStatus {
DRAFT
SENT
PAID
OVERDUE
CANCELLED
}

enum NotificationType {
RFQ_PUBLISHED
QUOTATION_RECEIVED
APPROVAL_REQUESTED
APPROVAL_GRANTED
APPROVAL_REJECTED
PO_ISSUED
INVOICE_SENT
VENDOR_REGISTERED
SYSTEM_ALERT
}

// ■■■ MODELS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

model User {
id String @id @default(cuid())
email String @unique
passwordHash String
firstName String
lastName String
role Role @default(PROCUREMENT_OFFICER)
department String?
avatarUrl String?
isActive Boolean @default(true)
isSuspended Boolean @default(false)
lastLoginAt DateTime?
inviteToken String? @unique
inviteExpiry DateTime?
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

vendor Vendor?
rfqsCreated RFQ[] @relation("CreatedBy")
approvals Approval[]
activityLogs ActivityLog[]
notifications Notification[]
loginHistory LoginHistory[]

@@index([email])
@@index([role])
@@map("users")
}

model Vendor {
```

```
id String @id @default(cuid())
userId String? @unique
companyName String
category String
gstNumber String? @unique
panNumber String?
contactEmail String
contactPhone String
website String?
address String?
country String @default("India")
logoUrl String?
status VendorStatus @default(PENDING_APPROVAL)
rating Float?
totalOrders Int @default(0)
totalSpend Decimal @default(0) @db.Decimal(18,2)
notes String?
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

user User? @relation(fields:[userId], references:[id])
rfqVendors RFQVendor[]
quotations Quotation[]

@@index([status])
@@index([category])
@@map("vendors")
}

model RFQ {
id String @id @default(cuid())
rfqNumber String @unique
title String
description String?
status RFQStatus @default(DRAFT)
deadline DateTime
createdById String
attachmentUrls String[]
tags String[]
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

createdBy User @relation("CreatedBy", fields:[createdById], references:[id])
items RFQItem[]
vendors RFQVendor[]
quotations Quotation[]

@@index([status])
@@index([deadline])
@@index([createdById])
@@map("rfqs")
}

model RFQItem {
id String @id @default(cuid())
rfqId String
itemName String
description String?
quantity Decimal @db.Decimal(10,2)
unit String
sortOrder Int @default(0)

rfq RFQ @relation(fields:[rfqId], references:[id], onDelete: Cascade)
@@map("rfq_items")
}

model RFQVendor {
rfqId String
vendorId String
invitedAt DateTime @default(now())
respondedAt DateTime?

rfq RFQ @relation(fields:[rfqId], references:[id], onDelete: Cascade)
vendor Vendor @relation(fields:[vendorId], references:[id])

@@id([rfqId, vendorId])
@@map("rfq_vendors")
}

model Quotation {
id String @id @default(cuid())
```

```
quotationNumber String @unique
rfqId String
vendorId String
status QuotationStatus @default(SUBMITTED)
deliveryDays Int
validUntil DateTime
notes String?
totalAmount Decimal @db.Decimal(18,2)
submittedAt DateTime @default(now())
updatedAt DateTime @updatedAt
version Int @default(1)

rfq RFQ @relation(fields:[rfqId], references:[id])
vendor Vendor @relation(fields:[vendorId], references:[id])
items QuotationItem[]
approval Approval?

@@index([rfqId])
@@index([vendorId])
@@index([status])
@@map("quotations")
}

model QuotationItem {
id String @id @default(cuid())
quotationId String
rfqItemId String
itemName String
quantity Decimal @db.Decimal(10,2)
unitPrice Decimal @db.Decimal(18,2)
taxRate Decimal @default(18) @db.Decimal(5,2)
totalPrice Decimal @db.Decimal(18,2)

quotation Quotation @relation(fields:[quotationId], references:[id], onDelete: Cascade)
@@map("quotation_items")
}

model Approval {
id String @id @default(cuid())
quotationId String @unique
status ApprovalStatus @default(PENDING)
approverId String
remarks String?
requestedAt DateTime @default(now())
resolvedAt DateTime?
level Int @default(1)

quotation Quotation @relation(fields:[quotationId], references:[id])
approver User @relation(fields:[approverId], references:[id])
purchaseOrder PurchaseOrder?

@@index([status])
@@index([approverId])
@@map("approvals")
}

model PurchaseOrder {
id String @id @default(cuid())
poNumber String @unique
approvalId String @unique
status POStatus @default(DRAFT)
subtotal Decimal @db.Decimal(18,2)
taxAmount Decimal @db.Decimal(18,2)
totalAmount Decimal @db.Decimal(18,2)
notes String?
deliveryDate DateTime?
issuedAt DateTime @default(now())
updatedAt DateTime @updatedAt

approval Approval @relation(fields:[approvalId], references:[id])
invoice Invoice?

@@index([status])
@@map("purchase_orders")
}

model Invoice {
id String @id @default(cuid())
invoiceNumber String @unique
poId String @unique
status InvoiceStatus @default(DRAFT)
```

```
subtotal Decimal @db.Decimal(18,2)
taxAmount Decimal @db.Decimal(18,2)
totalAmount Decimal @db.Decimal(18,2)
pdfUrl String?
dueDate DateTime
paidAt DateTime?
sentAt DateTime?
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

po PurchaseOrder @relation(fields:[poId], references:[id])
@@index([status])
@@map("invoices")
}

model ActivityLog {
id String @id @default(cuid())
userId String
action String
module String
entityId String?
entityType String?
metadata Json?
ipAddress String?
userAgent String?
createdAt DateTime @default(now())

user User @relation(fields:[userId], references:[id])
@@index([userId])
@@index([module])
@@index([createdAt])
@@map("activity_logs")
}

model Notification {
id String @id @default(cuid())
userId String
type NotificationType
title String
message String
isRead Boolean @default(false)
entityId String?
entityType String?
createdAt DateTime @default(now())

user User @relation(fields:[userId], references:[id], onDelete: Cascade)
@@index([userId, isRead])
@@map("notifications")
}

model LoginHistory {
id String @id @default(cuid())
userId String
ipAddress String?
userAgent String?
success Boolean
createdAt DateTime @default(now())

user User @relation(fields:[userId], references:[id], onDelete: Cascade)
@@index([userId])
@@map("login_history")
}
```

2.3 Indexes & Performance Optimization

In addition to Prisma-defined indexes, run these raw SQL statements for composite and partial indexes:

```
-- Composite index for vendor search by status + category
CREATE INDEX idx_vendors_status_category ON vendors(status, category);

-- Partial index for active RFQs only (most frequent query)
CREATE INDEX idx_rfqs_active ON rfqs(deadline) WHERE status = 'PUBLISHED';

-- Composite for notification polling (userId + unread + date desc)
CREATE INDEX idx_notif_user_unread ON notifications(user_id, is_read, created_at DESC);

-- Full-text search on vendor company name
CREATE INDEX idx_vendors_fts ON vendors USING gin(to_tsvector('english', company_name));

-- Activity log by module + entity for audit trails
```

```
CREATE INDEX idx_logs_module_entity ON activity_logs(module, entity_id, created_at DESC);
```

2.4 Seed Data Strategy

Seed the database with realistic test data for all roles and procurement states. Run: npx prisma db seed. Key seed items:

- 1 Admin user — admin@vendorbridge.com / Admin@123
- 2 Procurement Officers — officer1@vendorbridge.com / Officer@123
- 1 Manager/Approver — manager@vendorbridge.com / Manager@123
- 5 Vendor accounts (3 Active, 1 Pending, 1 Blacklisted) across categories: Hardware, Logistics, Software
- 3 RFQs (1 Draft, 1 Published with 2 quotations, 1 Closed with approved quotation)
- 2 Purchase Orders (1 Issued, 1 Delivered) each with a corresponding Invoice
- 20 Activity log entries across modules
- 10 Notifications (mix of read/unread) per Officer user

PART 3 — Authentication & Authorization (Screen 1)

JWT-based auth, RBAC middleware, password reset, and session handling

3.1 JWT Strategy & Token Architecture

VendorBridge uses a dual-token strategy: a short-lived access token (7 days, configurable) and a long-lived refresh token (30 days). Both are signed RS256 or HS256 depending on deployment scale. The access token carries role, userId, and email as claims.

```
// src/utils/jwt.util.ts
import jwt from 'jsonwebtoken';
import { env } from '../config/env';

export interface JwtPayload {
userId: string;
email: string;
role: string;
}

export const signAccess = (payload: JwtPayload) =>
jwt.sign(payload, env.JWT_SECRET, { expiresIn: env.JWT_EXPIRES_IN });

export const signRefresh = (payload: JwtPayload) =>
jwt.sign(payload, env.JWT_REFRESH_SECRET, { expiresIn: env.JWT_REFRESH_EXPIRES_IN });

export const verifyAccess = (token: string) =>
jwt.verify(token, env.JWT_SECRET) as JwtPayload;

export const verifyRefresh = (token: string) =>
jwt.verify(token, env.JWT_REFRESH_SECRET) as JwtPayload;
```

3.2 Auth API Endpoints

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	Public	Vendor self-registration (sends verification email)
POST	/api/v1/auth/login	Public	Email + password login → returns access + refresh to
POST	/api/v1/auth/refresh	Public (refresh token)	Exchange refresh token → new access token
POST	/api/v1/auth/logout	Protected	Invalidate refresh token (stored in Redis blacklist)
POST	/api/v1/auth/forgot-password	Public	Send password reset email with 15-min token
POST	/api/v1/auth/reset-password	Public (reset token)	Validate token + update password hash
POST	/api/v1/auth/verify-email	Public (email token)	Mark user email as verified
GET	/api/v1/auth/me	Protected	Return current user profile + role
PATCH	/api/v1/auth/change-password	Protected	Change own password (old + new required)

3.3 Role-Based Access Control (RBAC)

The roleGuard middleware is a factory that accepts allowed roles and returns a middleware function:

```
// src/middleware/roleGuard.ts
import { Role } from '@prisma/client';
import { AppError } from './errorHandler';

export const roleGuard = (...roles: Role[]) =>
```

```
(req: Request, res: Response, next: NextFunction) => {
if (!req.user) return next(new AppError('Unauthenticated', 401));
if (!roles.includes(req.user.role as Role))
return next(new AppError('Insufficient permissions', 403));
next();
};

// Usage in routes:
// router.post('/rfq', protect, roleGuard(Role.PROCUREMENT_OFFICER, Role.ADMIN),
createRFQ);
// router.patch('/approve', protect, roleGuard(Role.MANAGER, Role.ADMIN),
approveQuotation);
```

3.4 Password Reset Flow

• User POSTs /forgot-password with email. Backend checks if user exists (always returns 200 to prevent enumeration).

• If exists: generate a 32-byte random hex token, hash it with SHA-256, store hash + expiry (15 min) in Redis with key reset:userId.

• Send email with link: FRONTEND_URL/reset-password?token=

• User POSTs /reset-password with token + newPassword. Backend hashes the token, looks up Redis for a match, validates expiry.

• On success: update passwordHash in DB, delete Redis key, invalidate all refresh tokens for that user.

3.5 protect Middleware (JWT verification)

```
// src/middleware/protect.ts
export const protect = async (req, res, next) => {
const authHeader = req.headers.authorization;
if (!authHeader?.startsWith('Bearer '))
return next(new AppError('No token provided', 401));

const token = authHeader.split(' ')[1];

// Check Redis blacklist (logout invalidation)
const isBlacklisted = await redis.get(`blacklist:${token}`);
if (isBlacklisted) return next(new AppError('Token revoked', 401));

const payload = verifyAccess(token); // throws if invalid/expired

// Hydrate full user from DB (fresh role/suspension status)
const user = await prisma.user.findUnique({ where: { id: payload.userId } });
if (!user || !user.isActive || user.isSuspended)
return next(new AppError('Account inactive or suspended', 401));

req.user = user;
next();
};
```

PART 4 — Dashboard API (Screen 2)

KPI aggregation, analytics cards, quick-action counts, recent activity feed

4.1 Dashboard Endpoints

Method	Endpoint	Auth / Role	Returns
GET	/api/v1/dashboard/stats	All authenticated	KPI summary cards (5 key metrics)
GET	/api/v1/dashboard/pending-approvals	Officer / Manager	Count + list of pending approvals
GET	/api/v1/dashboard/active-rfqs	Officer / Admin	Active RFQ list with deadlines
GET	/api/v1/dashboard/recent-pos	Officer / Admin	Last 5 purchase orders
GET	/api/v1/dashboard/recent-invoices	Officer / Admin	Last 5 invoices with status
GET	/api/v1/dashboard/activity-feed	All authenticated	Latest 10 activity log entries

4.2 KPI Aggregation Service

The stats endpoint runs 5 parallel Prisma queries wrapped in Promise.all for performance:

```
// src/modules/dashboard/dashboard.service.ts
export const getDashboardStats = async (userId: string, role: string) => {
const [pendingApprovals, activeRFQs, totalVendors,
monthlySpend, pendingInvoices] = await Promise.all([

prisma.approval.count({ where: { status: 'PENDING' } }),

prisma.rFQ.count({ where: { status: 'PUBLISHED' } }),

prisma.vendor.count({ where: { status: 'ACTIVE' } }),

// Current month PO total
prisma.purchaseOrder.aggregate({
_sum: { totalAmount: true },
where: {
issuedAt: {
gte: new Date(new Date().getFullYear(), new Date().getMonth(), 1)
}
}
}),

prisma.invoice.count({ where: { status: { in: ['DRAFT', 'SENT'] } } })
]);

return { pendingApprovals, activeRFQs, totalVendors,
monthlySpend: monthlySpend._sum.totalAmount ?? 0,
pendingInvoices };
};
```

PART 5 — Vendor Management (Screen 3)

Full CRUD, GST validation, status lifecycle, search, filter, pagination

5.1 Vendor CRUD API Endpoints

Method	Endpoint	Role	Description
GET	/api/v1/vendors	All Auth	List vendors — paginated, searchable, filterable by stat
GET	/api/v1/vendors/:id	All Auth	Get single vendor detail with quotation history
POST	/api/v1/vendors	Admin	Create/register vendor (admin-initiated)
PATCH	/api/v1/vendors/:id	Admin / Officer	Update vendor fields (partial update)
PATCH	/api/v1/vendors/:id/status	Admin	Change status: ACTIVE / INACTIVE / BLACKLISTED
DELETE	/api/v1/vendors/:id	Admin	Soft-delete (set status to INACTIVE, never hard-delete)
POST	/api/v1/vendors/:id/logo	Admin	Upload vendor logo → Multer → S3 → update logoUrl
GET	/api/v1/vendors/categories	All Auth	Return list of distinct categories (for filter dropdown)

5.2 GST Validation Logic

Indian GST numbers follow the pattern: 2-digit state code + 10-char PAN + 1-digit entity + 1-digit checksum + 1 literal 'Z'. Validate with regex before saving:

```
// src/utils/gst.util.ts
export const GST_REGEX = /^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$/;

export const validateGST = (gst: string): boolean => {
if (!gst) return true; // GST is optional
return GST_REGEX.test(gst.toUpperCase());
};

// Tax calculation helper (GST breakdown)
export const calculateGST = (amount: number, rate: number = 18) => {
const taxAmount = (amount * rate) / 100;
return {
subtotal: amount,
cgst: taxAmount / 2, // 9% for intra-state
sgst: taxAmount / 2, // 9% for intra-state
igst: taxAmount, // 18% for inter-state (use one or the other)
total: amount + taxAmount
};
};
```

5.3 Vendor Status State Machine

From Status	To Status	Trigger	Who Can Perform
PENDING_APPROVAL	ACTIVE	Admin approves registration	Admin
PENDING_APPROVAL	INACTIVE	Admin rejects registration	Admin
ACTIVE	INACTIVE	Manual deactivation or inactivity rule	Admin
INACTIVE	ACTIVE	Re-activation	Admin

ACTIVE	BLACKLISTED	Policy violation	Admin
BLACKLISTED	ACTIVE	Appeal approved	Admin (manual)

5.4 Search, Filter & Pagination

```
// GET /api/v1/vendors?search=apex&status=ACTIVE&category=Hardware&page=1&limit=20

export const listVendors = async (query: VendorQueryDto) => {
const { search, status, category, page = 1, limit = 20, sortBy = 'createdAt', order =
'desc' } = query;

const where: Prisma.VendorWhereInput = {
...(status && { status }),
...(category && { category }),
...(search && {
OR: [
{ companyName: { contains: search, mode: 'insensitive' } },
{ contactEmail: { contains: search, mode: 'insensitive' } },
{ gstNumber: { contains: search, mode: 'insensitive' } },
]
}),
status: { not: 'BLACKLISTED' } // exclude blacklisted unless admin
};

const [data, total] = await Promise.all([
prisma.vendor.findMany({
where,
skip: (page - 1) * limit,
take: limit,
orderBy: { [sortBy]: order },
include: { _count: { select: { quotations: true } } }
}),
prisma.vendor.count({ where })
]);

return { data, total, page, limit, totalPages: Math.ceil(total / limit) };
};
```

PART 6 — RFQ Module (Screen 4)

RFQ creation, lifecycle, file attachments, vendor assignment, and invitation emails

6.1 RFQ API Endpoints

Method	Endpoint	Role	Description
GET	/api/v1/rfqs	All Auth	List RFQs with pagination, search, status filter
GET	/api/v1/rfqs/:id	All Auth	Get full RFQ detail including items and assigned vend
POST	/api/v1/rfqs	Officer / Admin	Create new RFQ (DRAFT state) with items array
PATCH	/api/v1/rfqs/:id	Officer / Admin	Update RFQ metadata and items (DRAFT only)
DELETE	/api/v1/rfqs/:id	Officer / Admin	Soft-cancel RFQ (only if in DRAFT or PUBLISHED)
POST	/api/v1/rfqs/:id/publish	Officer / Admin	Transition DRAFT → PUBLISHED, trigger vendor em
POST	/api/v1/rfqs/:id/close	Officer / Admin	Transition PUBLISHED → CLOSED
POST	/api/v1/rfqs/:id/attachments	Officer / Admin	Upload attachment files (up to 5, max 10MB each)
DELETE	/api/v1/rfqs/:id/attachments/:fileKey	Officer / Admin	Remove an attachment from S3
POST	/api/v1/rfqs/:id/vendors	Officer / Admin	Assign/invite additional vendors
GET	/api/v1/rfqs/:id/quotations	Officer / Admin	List all quotations submitted for this RFQ

6.2 RFQ Auto-Number Generation

```
// Generate sequential RFQ number: RFQ-2024-0001
export const generateRFQNumber = async (): Promise<string> => {
const year = new Date().getFullYear();
const prefix = `RFQ-${year}-`;
const last = await prisma.rFQ.findFirst({
where: { rfqNumber: { startsWith: prefix } },
orderBy: { rfqNumber: 'desc' }
});
const nextSeq = last
? parseInt(last.rfqNumber.split('-')[2]) + 1
: 1;
return `${prefix}${String(nextSeq).padStart(4, '0')}`;
};
```

6.3 File Attachment Handling

Configure Multer with S3 storage for file attachments on RFQs:

```
// src/middleware/upload.ts
import multer from 'multer';
import multerS3 from 'multer-s3';
import { s3Client } from '../config/s3';

export const rfqUpload = multer({
storage: multerS3({
s3: s3Client,
bucket: env.AWS_S3_BUCKET,
key: (req, file, cb) => {
const key = `rfq-attachments/${req.params.id}/${Date.now()}-${file.originalname}`;
cb(null, key);
},
contentType: multerS3.AUTO_CONTENT_TYPE,
```

```
}),
limits: { fileSize: 10 * 1024 * 1024, files: 5 },
fileFilter: (req, file, cb) => {
const allowed = ['image/jpeg','image/png','application/pdf',
'application/vnd.ms-excel',
'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'];
cb(null, allowed.includes(file.mimetype));
}
});
```

6.4 Vendor Assignment & Email Invitation

On publish, iterate over assigned vendors and send each an invitation email with a secure one-time link containing their vendor token:

```
export const publishRFQ = async (rfqId: string, actorId: string) => {
const rfq = await prisma.rFQ.update({
where: { id: rfqId },
data: { status: 'PUBLISHED' },
include: { vendors: { include: { vendor: { include: { user: true } } } } } }
});

// Send invitation emails to all assigned vendors
const emailPromises = rfq.vendors.map(rv => {
if (!rv.vendor.user?.email) return Promise.resolve();
return sendEmail({
to: rv.vendor.user.email,
subject: `New RFQ Invitation: ${rfq.title}`,
template: 'rfq-invitation',
data: {
vendorName: rv.vendor.companyName,
rfqNumber: rfq.rfqNumber,
rfqTitle: rfq.title,
deadline: rfq.deadline,
link: `${env.FRONTEND_URL}/rfqs/${rfqId}`,
}
});
});
await Promise.allSettled(emailPromises);

// Log activity
await writeAuditLog(actorId, 'RFQ_PUBLISHED', 'rfq', rfqId, { title: rfq.title });
return rfq;
};
```

PART 7 — Vendor Quotation Submission (Screen 5)

Quotation API, versioning, business validation rules

7.1 Quotation API Endpoints

Method	Endpoint	Role	Description
GET	/api/v1/quotations	Officer / Admin	List all quotations (filterable by rfqId, status, vendorI
GET	/api/v1/quotations/:id	Officer / Vendor / Admin	Get quotation detail with items breakdown
POST	/api/v1/quotations	Vendor	Submit new quotation for an open RFQ
PATCH	/api/v1/quotations/:id	Vendor	Edit own quotation (only if SUBMITTED, before dea
DELETE	/api/v1/quotations/:id	Vendor / Admin	Withdraw quotation (soft status = REJECTED)
PATCH	/api/v1/quotations/:id/shortlist	Officer / Admin	Shortlist quotation for approval workflow

7.2 Quotation Submission Validation

• Vendor must be assigned to the RFQ (exists in rfq_vendors pivot table).

• RFQ status must be PUBLISHED. Reject if CLOSED or CANCELLED.

• Deadline must not have passed (rfq.deadline > now()).

• One active quotation per vendor per RFQ. If re-submitting, increment version and mark old quotation as superseded.

• Every rfq_item must have a corresponding quotation_item. Line items validated against rfqItemId.

• unitPrice must be > 0. Quantity must match or be >= rfqItem quantity.

• totalPrice for each line = unitPrice * quantity * (1 + taxRate/100). Auto-calculate; reject if frontend sends incorrect totals.

• quotation.totalAmount must equal sum of all line item totalPrice values.

• validUntil must be at least 30 days in the future.

7.3 Auto-Total Calculation on Server

```
// Always recalculate totals server-side to prevent manipulation
const items = dto.items.map(item => ({
...item,
totalPrice: item.unitPrice * item.quantity * (1 + item.taxRate / 100)
}));

const totalAmount = items.reduce((sum, i) => sum + i.totalPrice, 0);

const quotation = await prisma.quotation.create({
data: {
...quotationFields,
totalAmount,
items: { create: items }
}
});
```

PART 8 — Quotation Comparison Engine (Screen 6)

Side-by-side comparison API, lowest-price detection, scoring algorithm

8.1 Comparison API Endpoint

A single endpoint returns all quotations for an RFQ in a comparison-ready format:

```
// GET /api/v1/rfqs/:rfqId/compare
// Returns: rfq items, all quotations side by side, winners per item, overall winner

export const compareQuotations = async (rfqId: string) => {
const rfq = await prisma.rFQ.findUnique({
where: { id: rfqId },
include: {
items: true,
quotations: {
where: { status: { in: ['SUBMITTED','SHORTLISTED','UNDER_REVIEW'] } },
include: { items: true, vendor: true }
}
}
});

// Build comparison matrix
const matrix = rfq!.items.map(rfqItem => ({
rfqItem,
bids: rfq!.quotations.map(q => {
const qi = q.items.find(i => i.rfqItemId === rfqItem.id);
return {
quotationId: q.id,
vendorName: q.vendor.companyName,
vendorRating: q.vendor.rating,
unitPrice: qi?.unitPrice ?? null,
totalPrice: qi?.totalPrice ?? null,
isLowest: false // computed below
};
})
}));

// Mark lowest price per item
matrix.forEach(row => {
const prices = row.bids.map(b => b.totalPrice).filter(Boolean);
const min = Math.min(...prices as number[]);
row.bids.forEach(b => { b.isLowest = b.totalPrice === min; });
});

// Score each quotation (see 8.3)
const scored = rfq!.quotations.map(q => ({
quotationId: q.id,
vendorId: q.vendorId,
vendorName: q.vendor.companyName,
totalAmount: q.totalAmount,
deliveryDays: q.deliveryDays,
rating: q.vendor.rating,
score: computeScore(q, rfq!.quotations)
})).sort((a,b) => b.score - a.score);

return { rfq, matrix, scored };
};
```

8.2 Scoring Algorithm

The scoring algorithm weighs price (50%), delivery speed (30%), and vendor rating (20%). Scores are normalized 0–100 within the set of competing quotations.

```
const computeScore = (q: QuotationWithVendor, all: QuotationWithVendor[]): number => {
const prices = all.map(x => Number(x.totalAmount));
const minPrice = Math.min(...prices);
const maxPrice = Math.max(...prices);
const deliveries = all.map(x => x.deliveryDays);
const minDelivery = Math.min(...deliveries);
const maxDelivery = Math.max(...deliveries);
```

```
const priceScore = maxPrice === minPrice ? 100
: 100 * (1 - (Number(q.totalAmount) - minPrice) / (maxPrice - minPrice));

const deliveryScore = maxDelivery === minDelivery ? 100
: 100 * (1 - (q.deliveryDays - minDelivery) / (maxDelivery - minDelivery));

const ratingScore = ((q.vendor.rating ?? 3) / 5) * 100;

return (priceScore * 0.5) + (deliveryScore * 0.3) + (ratingScore * 0.2);
};
```

PART 9 — Approval Workflow Engine (Screen 7)

State machine, approve/reject API, multi-level chains, notification dispatch

9.1 Approval State Machine

From State	To State	Trigger	Side Effects
—	PENDING	Procurement Officer shortlists a quotation	Notify assigned approver(s)
PENDING	APPROVED	Manager calls /approve	Create PurchaseOrder, notify Officer
PENDING	REJECTED	Manager calls /reject with remarks	Notify Officer + Vendor
PENDING	ESCALATED	Auto-escalate after 48h SLA breach	Notify next-level approver
ESCALATED	APPROVED	Senior Manager approves	Same as APPROVED above
ESCALATED	REJECTED	Senior Manager rejects	Same as REJECTED above

9.2 Approval API Endpoints

Method	Endpoint	Role	Description
GET	/api/v1/approvals	Manager / Admin	List approvals — filterable by status, date range
GET	/api/v1/approvals/:id	Manager / Officer / Admin	Get approval detail with timeline
POST	/api/v1/approvals	Officer / Admin	Create approval request (shortlist a quotation)
POST	/api/v1/approvals/:id/approve	Manager / Admin	Approve → auto-create PO
POST	/api/v1/approvals/:id/reject	Manager / Admin	Reject with mandatory remarks field
GET	/api/v1/approvals/my-queue	Manager	Approvals pending action from requesting user

9.3 Approve Action — Auto PO Creation

```
export const approveQuotation = async (approvalId: string, approverId: string, remarks?:
string) => {
const approval = await prisma.approval.findUnique({
where: { id: approvalId },
include: { quotation: { include: { items: true, vendor: true } } }
});

if (!approval || approval.status !== 'PENDING')
throw new AppError('Approval not found or not pending', 400);

if (approval.approverId !== approverId)
throw new AppError('Not authorized to approve this request', 403);

const poNumber = await generatePONumber();
const q = approval.quotation;

// Transaction: update approval + create PO atomically
const [updatedApproval, po] = await prisma.$transaction([
prisma.approval.update({
where: { id: approvalId },
data: { status: 'APPROVED', remarks, resolvedAt: new Date() }
}),
prisma.purchaseOrder.create({
data: {
```

```
poNumber,
approvalId,
status: 'ISSUED',
subtotal: q.totalAmount,
taxAmount: calculateTaxAmount(q.items),
totalAmount: q.totalAmount,
}
}),
// Update quotation status
prisma.quotation.update({
where: { id: q.id },
data: { status: 'ACCEPTED' }
}),
]);

// Notify relevant parties
await Promise.allSettled([
notifyUser(q.rfq.createdById, 'APPROVAL_GRANTED', { poNumber }),
notifyVendor(q.vendorId, 'PO_ISSUED', { poNumber }),
writeAuditLog(approverId, 'APPROVAL_GRANTED', 'approval', approvalId)
]);

return { approval: updatedApproval, po };
};
```

PART 10 — Purchase Orders & Invoice Generation (Screen 8)

Auto-PO numbers, invoice CRUD, PDF generation, print, email dispatch

10.1 Purchase Order & Invoice Endpoints

Method	Endpoint	Role	Description
GET	/api/v1/purchase-orders	Officer / Admin	List POs with status, date filters, pagination
GET	/api/v1/purchase-orders/:id	Officer / Admin	Full PO detail with vendor, items, invoice status
PATCH	/api/v1/purchase-orders/:id/status	Officer / Admin	Update PO status: ACKNOWLEDGED/DELIVERED
GET	/api/v1/invoices	Officer / Admin	List all invoices with status filter
GET	/api/v1/invoices/:id	Officer / Admin	Invoice detail with line items and tax breakdown
POST	/api/v1/invoices	Officer / Admin	Generate invoice from PO (auto-populate from quotat
GET	/api/v1/invoices/:id/pdf	Officer / Admin	Stream invoice PDF (generate on-demand or serve fr
POST	/api/v1/invoices/:id/send-email	Officer / Admin	Email invoice PDF to vendor contact
PATCH	/api/v1/invoices/:id/status	Officer / Admin	Mark PAID, OVERDUE, CANCELLED

10.2 Auto-Number Generation for PO & Invoice

```
// PO: VB-PO-2024-00001 | Invoice: VB-INV-2024-00001
export const generatePONumber = async () => generateSequentialNumber('PO');
export const generateInvoiceNumber = async () => generateSequentialNumber('INV');

const generateSequentialNumber = async (prefix: 'PO' | 'INV') => {
const year = new Date().getFullYear();
const key = `VB-${prefix}-${year}-`;
const model = prefix === 'PO' ? 'purchaseOrder' : 'invoice';
const field = prefix === 'PO' ? 'poNumber' : 'invoiceNumber';

// Use Redis atomic counter for race-condition-safe incrementing
const seq = await redis.incr(`counter:${key}`);
// Set expiry to 13 months to allow clean year rollover
await redis.expire(`counter:${key}`, 13 * 30 * 24 * 3600);
return `${key}${String(seq).padStart(5, '0')}`;
};
```

10.3 GST Tax Calculation Service

```
export const buildInvoiceTaxBreakdown = (items: QuotationItem[], isIntraState: boolean) =>
{
const subtotal = items.reduce((s, i) => s + Number(i.totalPrice), 0);
const taxableItems = items.map(i => ({
...i,
taxRate: Number(i.taxRate),
taxableAmount: Number(i.unitPrice) * Number(i.quantity),
taxAmount: Number(i.totalPrice) - (Number(i.unitPrice) * Number(i.quantity))
}));

const totalTax = taxableItems.reduce((s,i) => s + i.taxAmount, 0);

return {
subtotal,
cgst: isIntraState ? totalTax / 2 : 0,
sgst: isIntraState ? totalTax / 2 : 0,
```

```
igst: isIntraState ? 0 : totalTax,
totalTax,
grandTotal: subtotal + totalTax,
items: taxableItems
};
};
```

10.4 PDF Invoice Generation (PDFKit)

```
// src/utils/pdf.util.ts
import PDFDocument from 'pdfkit';
import { PassThrough } from 'stream';

export const generateInvoicePDF = async (invoice: InvoiceWithDetails): Promise<Buffer> =>
{
return new Promise((resolve, reject) => {
const doc = new PDFDocument({ size: 'A4', margin: 50 });
const buffers: Buffer[] = [];
const pass = new PassThrough();

doc.pipe(pass);
pass.on('data', chunk => buffers.push(chunk));
pass.on('end', () => resolve(Buffer.concat(buffers)));
pass.on('error', reject);

// ■■ Header ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
doc.rect(0, 0, 595, 80).fill('#0154CC');
doc.fillColor('white').fontSize(22).font('Helvetica-Bold')
.text('VendorBridge', 50, 25);
doc.fontSize(10).font('Helvetica')
.text('Enterprise Procurement ERP', 50, 50);
doc.fillColor('#1C1B1B').text('TAX INVOICE', 450, 30,
{ width: 100, align: 'right' }).fontSize(12);

// ■■ Invoice Meta ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
doc.fontSize(10).moveDown(3);
doc.text(`Invoice No: ${invoice.invoiceNumber}`, 50);
doc.text(`Date: ${formatDate(invoice.createdAt)}`, 50);
doc.text(`Due: ${formatDate(invoice.dueDate)}`, 50);
doc.text(`PO Ref: ${invoice.po.poNumber}`, 50);

// ■■ Vendor Info ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
doc.text('Bill To:', 350, doc.y - 60);
doc.text(invoice.po.approval.quotation.vendor.companyName, 350);
doc.text(invoice.po.approval.quotation.vendor.contactEmail, 350);

// ■■ Items Table ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
drawInvoiceTable(doc, invoice.items);

// ■■ Totals ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const tax = buildInvoiceTaxBreakdown(invoice.items, true);
doc.text(`Subtotal: ■${tax.subtotal.toFixed(2)}`, 350);
doc.text(`CGST (9%): ■${tax.cgst.toFixed(2)}`, 350);
doc.text(`SGST (9%): ■${tax.sgst.toFixed(2)}`, 350);
doc.fontSize(12).font('Helvetica-Bold')
.text(`TOTAL: ■${tax.grandTotal.toFixed(2)}`, 350);

doc.end();
});
};
```

10.5 Send Invoice via Email

```
export const sendInvoiceEmail = async (invoiceId: string) => {
const invoice = await getInvoiceWithDetails(invoiceId);
const pdfBuffer = await generateInvoicePDF(invoice);

await sendEmail({
to: invoice.po.approval.quotation.vendor.contactEmail,
subject: `Invoice ${invoice.invoiceNumber} from VendorBridge`,
template: 'invoice-email',
data: { invoiceNumber: invoice.invoiceNumber, total: invoice.totalAmount },
attachments: [{
filename: `${invoice.invoiceNumber}.pdf`,
content: pdfBuffer,
contentType: 'application/pdf'
}]
});
```

```
await prisma.invoice.update({
where: { id: invoiceId },
data: { status: 'SENT', sentAt: new Date() }
});
};
```

PART 11 — Activity Logs & Notifications (Screen 9)

Audit log service, real-time Socket.io notifications, notification preferences

11.1 Audit Log Service

The writeAuditLog utility is called from every service function that mutates data. It writes to activity_logs asynchronously (non-blocking via setImmediate).

```
// src/utils/audit.util.ts
export const writeAuditLog = (
userId: string,
action: string,
module: string,
entityId?: string,
metadata?: Record<string, unknown>
) => {
// Non-blocking — don't await in caller
setImmediate(async () => {
await prisma.activityLog.create({
data: { userId, action, module, entityId,
entityType: module, metadata }
}).catch(err => logger.error('AuditLog write failed', err));
});
};
```

11.2 Activity Log Endpoints

Method	Endpoint	Role	Description
GET	/api/v1/activity-logs	Admin / Officer	List logs — filter by module, user, date range, action
GET	/api/v1/activity-logs/my	All Auth	Current user's own activity logs
GET	/api/v1/notifications	All Auth	User's notifications — filter unread, paginated
PATCH	/api/v1/notifications/:id/read	All Auth	Mark single notification as read
PATCH	/api/v1/notifications/read-all	All Auth	Mark all notifications as read for current user
GET	/api/v1/notifications/unread-count	All Auth	Returns integer count of unread notifications (for bad

11.3 Real-Time Notifications (Socket.io)

```
// src/server.ts
import { Server as SocketIOServer } from 'socket.io';
import { verifyAccess } from './utils/jwt.util';

export let io: SocketIOServer;

export const initSocketIO = (httpServer: any) => {
io = new SocketIOServer(httpServer, {
cors: { origin: env.FRONTEND_URL, credentials: true }
});

// Auth middleware for socket connections
io.use((socket, next) => {
const token = socket.handshake.auth.token;
if (!token) return next(new Error('Unauthorized'));
const payload = verifyAccess(token);
socket.data.userId = payload.userId;
next();
});

io.on('connection', (socket) => {
// Join personal room by userId
```

```
socket.join(`user:${socket.data.userId}`);
});
};

// Push notification from anywhere in the app:
// io.to(`user:${userId}`).emit('notification', { type, title, message });

// src/utils/notify.util.ts
export const notifyUser = async (userId: string, type: NotificationType,
data: Record<string, unknown>) => {
const notification = await prisma.notification.create({
data: { userId, type, title: getTitle(type, data), message: getMessage(type, data) }
});
io.to(`user:${userId}`).emit('notification', notification);
};
```

PART 12 — Reports & Analytics (Screen 10)

Analytics aggregation queries, vendor scoring, exportable reports (PDF/XLSX/CSV)

12.1 Analytics Endpoints

Method	Endpoint	Role	Description
GET	/api/v1/reports/summary	Admin / Officer	KPI cards: total spend, active RFQs, PO rate, avg ap
GET	/api/v1/reports/spending-trend	Admin / Officer	Monthly spending data for bar/line chart (12 months)
GET	/api/v1/reports/spending-by-category	Admin / Officer	Category-wise spend breakdown for donut chart
GET	/api/v1/reports/vendor-performance	Admin	Ranked vendor list with scores, spend, PO count, rati
GET	/api/v1/reports/rfq-funnel	Admin / Officer	RFQ → Quotation → Approval → PO → Invoice conv
GET	/api/v1/reports/budget-utilization	Admin	Budget vs actual spend by department/category
GET	/api/v1/reports/approval-turnaround	Admin / Manager	Average approval time trend chart data
GET	/api/v1/reports/export	Admin / Officer	Generate and stream exportable report (PDF/XLSX/C

12.2 Monthly Spending Trend Query

```
// Prisma raw query for monthly spend grouped by month
export const getSpendingTrend = async (months: number = 12) => {
const startDate = new Date();
startDate.setMonth(startDate.getMonth() - months + 1);
startDate.setDate(1);

const result = await prisma.$queryRaw<MonthlySpend[]>`
SELECT
DATE_TRUNC('month', issued_at)::date AS month,
SUM(total_amount)::float AS po_total,
COUNT(*)::int AS po_count
FROM purchase_orders
WHERE issued_at >= ${startDate}
AND status != 'CANCELLED'
GROUP BY 1
ORDER BY 1 ASC
`;

// Fill gaps (months with zero POs)
return fillMonthlyGaps(result, startDate, new Date());
};
```

12.3 Vendor Performance Scoring Query

```
export const getVendorPerformance = async () => {
const vendors = await prisma.vendor.findMany({
where: { status: 'ACTIVE' },
include: {
quotations: {
where: { status: 'ACCEPTED' },
include: {
approval: { include: { purchaseOrder: true } }
}
}
}
});

return vendors.map(v => {
const acceptedPOs = v.quotations.filter(q => q.approval?.purchaseOrder);
const totalSpend = acceptedPOs.reduce((s, q) =>
```

```
s + Number(q.approval!.purchaseOrder!.totalAmount), 0);

// On-time delivery rate = POs marked DELIVERED on time / total POs
const deliveryRate = computeDeliveryRate(acceptedPOs);

// Composite score
const score = ((v.rating ?? 3) / 5 * 40) +
(deliveryRate * 40) +
(Math.min(acceptedPOs.length / 50, 1) * 20);

return { vendor: v, totalSpend, poCount: acceptedPOs.length,
deliveryRate, score: Math.round(score) };
}).sort((a, b) => b.score - a.score);
};
```

12.4 Export Reports (XLSX / CSV / PDF)

```
// GET
/api/v1/reports/export?type=vendor-performance&format=xlsx&from=2024-01-01&to=2024-12-31

export const exportReport = async (dto: ExportReportDto, res: Response) => {
const data = await getReportData(dto.type, dto.from, dto.to);

if (dto.format === 'xlsx') {
const wb = xlsx.utils.book_new();
const ws = xlsx.utils.json_to_sheet(data);
xlsx.utils.book_append_sheet(wb, ws, 'Report');
const buf = xlsx.write(wb, { type: 'buffer', bookType: 'xlsx' });
res.set({ 'Content-Type': 'application/vnd.openxmlformats...',
'Content-Disposition': `attachment; filename="${dto.type}.xlsx"` });
return res.send(buf);
}

if (dto.format === 'csv') {
const csv = await csvStringify(data, { header: true });
res.set({ 'Content-Type': 'text/csv',
'Content-Disposition': `attachment; filename="${dto.type}.csv"` });
return res.send(csv);
}

if (dto.format === 'pdf') {
const pdfBuffer = await generateReportPDF(dto.type, data);
res.set({ 'Content-Type': 'application/pdf',
'Content-Disposition': `attachment; filename="${dto.type}.pdf"` });
return res.send(pdfBuffer);
}
};
```

PART 13 — User Management & Admin Panel (Screen 11)

Admin CRUD, role assignment, invite flow, login activity tracking

13.1 User Management Endpoints

Method	Endpoint	Role	Description
GET	/api/v1/users	Admin	List all users — filter by role, status, department, search
GET	/api/v1/users/:id	Admin	Get single user with role, login history, activity summary
PATCH	/api/v1/users/:id	Admin	Update user details: name, department, role
PATCH	/api/v1/users/:id/role	Admin	Change user role (logs transition + sends email)
PATCH	/api/v1/users/:id/suspend	Admin	Suspend account — prevents login, invalidates tokens
PATCH	/api/v1/users/:id/activate	Admin	Re-activate suspended account
DELETE	/api/v1/users/:id	Admin	Soft-delete: mark isActive=false (never hard-delete)
POST	/api/v1/users/invite	Admin	Send invitation email with one-time setup link
GET	/api/v1/users/stats	Admin	Aggregate user stats: total, active, pending, suspended
GET	/api/v1/users/:id/login-history	Admin	Paginated login history with IP + user-agent

13.2 Invite User Flow

• Admin POSTs /users/invite with { email, role, firstName, lastName, department }.

• Backend checks email not already registered. Creates User with isActive=false, inviteToken=uuid(), inviteExpiry=now()+48h.

• Send email: 'You've been invited to VendorBridge. Set up your account: FRONTEND_URL/accept-invite?token=XXX'

• Frontend POSTs /auth/accept-invite with { token, newPassword }. Backend validates token, sets password, isActive=true, clears inviteToken.

• If invite token expired (>48h): admin must re-send via POST /users/:id/resend-invite.

13.3 Account Suspension

```
export const suspendUser = async (targetId: string, adminId: string) => {
if (targetId === adminId) throw new AppError('Cannot suspend yourself', 400);

await prisma.user.update({
where: { id: targetId },
data: { isSuspended: true }
});

// Invalidate all active refresh tokens for this user
// Pattern: revoke:userId → set in Redis with 30d TTL
await redis.set(`revoke:${targetId}`, '1', { EX: 30 * 24 * 3600 });

// protect middleware checks for revoke:{userId} key on every request
await writeAuditLog(adminId, 'USER_SUSPENDED', 'users', targetId);
};
```

PART 14 — Cross-Cutting Concerns

Error handling, validation, rate limiting, email service, Redis caching, security

14.1 Global Error Handler

```
// src/middleware/errorHandler.ts
export class AppError extends Error {
statusCode: number;
isOperational: boolean;
constructor(message: string, statusCode: number) {
super(message);
this.statusCode = statusCode;
this.isOperational = true;
}
}

export const errorHandler = (err: any, req: Request, res: Response, next: NextFunction) =>
{
const statusCode = err.statusCode || 500;
const message = err.isOperational ? err.message : 'Internal server error';

// Prisma known error codes
if (err.code === 'P2002') return res.status(409).json(
{ status: 'fail', message: 'Duplicate entry: record already exists' });
if (err.code === 'P2025') return res.status(404).json(
{ status: 'fail', message: 'Record not found' });

logger.error({ err, path: req.path, method: req.method });
res.status(statusCode).json({ status: statusCode < 500 ? 'fail' : 'error', message });
};
```

14.2 Request Validation (Zod)

```
// src/middleware/validate.ts
import { ZodSchema } from 'zod';

export const validate = (schema: ZodSchema, target: 'body' | 'query' | 'params' = 'body')
=>
(req: Request, res: Response, next: NextFunction) => {
const result = schema.safeParse(req[target]);
if (!result.success) {
const errors = result.error.flatten().fieldErrors;
return next(new AppError(`Validation failed: ${JSON.stringify(errors)}`, 422));
}
req[target] = result.data; // replace with sanitized data
next();
};

// Example Zod schema for create vendor:
export const createVendorSchema = z.object({
companyName: z.string().min(2).max(100),
category: z.enum(['Hardware','Software','Logistics','Services','Manufacturing']),
gstNumber: z.string().regex(GST_REGEX).optional(),
contactEmail: z.string().email(),
contactPhone: z.string().min(10).max(15),
website: z.string().url().optional(),
country: z.string().default('India'),
});
```

14.3 Rate Limiting Configuration

```
// src/middleware/rateLimiter.ts
import rateLimit from 'express-rate-limit';

// Strict limit for auth endpoints
export const authLimiter = rateLimit({
windowMs: 15 * 60 * 1000, // 15 minutes
max: 10, // 10 attempts per window
message: 'Too many login attempts. Please try again in 15 minutes.',
standardHeaders: true,
legacyHeaders: false,
```

```
});

// General API limiter
export const apiLimiter = rateLimit({
windowMs: 60 * 1000, // 1 minute
max: 100, // 100 requests per minute
skipSuccessfulRequests: true,
});

// File upload limiter
export const uploadLimiter = rateLimit({
windowMs: 60 * 60 * 1000, // 1 hour
max: 50,
});
```

14.4 Email Service (Nodemailer Wrapper)

```
// src/utils/email.util.ts
import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransport({
host: env.SMTP_HOST, port: env.SMTP_PORT,
auth: { user: env.SMTP_USER, pass: env.SMTP_PASS }
});

interface EmailOptions {
to: string;
subject: string;
template: EmailTemplate;
data: Record<string, unknown>;
attachments?: Attachment[];
}

export const sendEmail = async (opts: EmailOptions) => {
const html = renderTemplate(opts.template, opts.data);
await transporter.sendMail({
from: `VendorBridge <${env.EMAIL_FROM}>`,
to: opts.to,
subject: opts.subject,
html,
attachments: opts.attachments
});
logger.info(`Email sent to ${opts.to} [${opts.template}]`);
};
```

14.5 Redis Caching Strategy

Cache Key Pattern	TTL	What is Cached
dashboard:stats:{userId}	2 min	Dashboard KPI aggregates — invalidate on any PO/RFQ/approval change
vendor:list:{hash}	5 min	Vendor list query results (hash = query params hash)
vendor:{id}	10 min	Single vendor detail — invalidate on vendor update
reports:trend:{year}	30 min	Monthly spending trend — heavy query
reports:vendor-perf	15 min	Vendor performance ranking table
rfq:compare:{rfqId}	5 min	Quotation comparison matrix — invalidate on new quotation
counter:VB-PO-{year}	13 months	Atomic PO number sequence counter
blacklist:{token}	7 days	Revoked JWT access tokens
revoke:{userId}	30 days	Full user token revocation (suspension)
reset:{userId}	15 min	Password reset token hash

PART 15 — Deployment & DevOps

Docker setup, environment strategy, health checks, production notes

15.1 Dockerfile

```
# docker/Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx prisma generate
RUN npm run build

FROM node:20-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/prisma ./prisma
COPY --from=builder /app/package.json .

# Install Chromium for Puppeteer PDF generation
RUN apk add --no-cache chromium
ENV PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium-browser

EXPOSE 5000
CMD ["node", "dist/server.js"]
```

15.2 Docker Compose (docker-compose.yml)

```
version: '3.9'
services:
api:
build:
context: .
dockerfile: docker/Dockerfile
ports:
- '5000:5000'
env_file: .env
depends_on:
postgres:
condition: service_healthy
redis:
condition: service_healthy
restart: unless-stopped

postgres:
image: postgres:15-alpine
environment:
POSTGRES_DB: vendorbridge
POSTGRES_USER: postgres
POSTGRES_PASSWORD: postgres
volumes:
- postgres_data:/var/lib/postgresql/data
healthcheck:
test: ['CMD-SHELL', 'pg_isready -U postgres']
interval: 5s
retries: 5
ports:
- '5432:5432'

redis:
image: redis:7-alpine
command: redis-server --requirepass redis_password
volumes:
- redis_data:/data
healthcheck:
test: ['CMD', 'redis-cli', 'ping']
interval: 5s
retries: 5
ports:
- '6379:6379'
```

```
volumes:
postgres_data:
redis_data:
```

15.3 Health Check Endpoint

```
// GET /api/v1/health
router.get('/health', async (req, res) => {
const checks = await Promise.allSettled([
prisma.$queryRaw`SELECT 1`, // DB ping
redis.ping(), // Redis ping
]);

const db = checks[0].status === 'fulfilled';
const cache = checks[1].status === 'fulfilled';

res.status(db && cache ? 200 : 503).json({
status: db && cache ? 'ok' : 'degraded',
timestamp: new Date().toISOString(),
services: { database: db ? 'up' : 'down', redis: cache ? 'up' : 'down' }
});
});
```

15.4 Complete API Route Registration (app.ts)

```
// src/app.ts
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import { apiLimiter } from './middleware/rateLimiter';
import { errorHandler } from './middleware/errorHandler';

// Route imports
import authRoutes from './modules/auth/auth.routes';
import dashboardRoutes from './modules/dashboard/dashboard.routes';
import vendorRoutes from './modules/vendors/vendors.routes';
import rfqRoutes from './modules/rfq/rfq.routes';
import quotationRoutes from './modules/quotations/quotations.routes';
import approvalRoutes from './modules/approvals/approvals.routes';
import poRoutes from './modules/purchase-orders/po.routes';
import invoiceRoutes from './modules/invoices/invoices.routes';
import activityRoutes from './modules/activity-logs/activity.routes';
import reportRoutes from './modules/reports/reports.routes';
import userRoutes from './modules/users/users.routes';
import healthRoutes from './modules/health/health.routes';

const app = express();
const API = '/api/v1';

app.use(helmet());
app.use(cors({ origin: env.FRONTEND_URL, credentials: true }));
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));
app.use(API, apiLimiter);

app.use(`${API}/health`, healthRoutes);
app.use(`${API}/auth`, authRoutes);
app.use(`${API}/dashboard`, dashboardRoutes);
app.use(`${API}/vendors`, vendorRoutes);
app.use(`${API}/rfqs`, rfqRoutes);
app.use(`${API}/quotations`, quotationRoutes);
app.use(`${API}/approvals`, approvalRoutes);
app.use(`${API}/purchase-orders`, poRoutes);
app.use(`${API}/invoices`, invoiceRoutes);
app.use(`${API}/activity-logs`, activityRoutes);
app.use(`${API}/reports`, reportRoutes);
app.use(`${API}/users`, userRoutes);

// Swagger docs
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));

// Global error handler (must be last)
app.use(errorHandler);

export default app;
```

15.5 Standard API Response Format

All API responses follow a consistent envelope format for frontend consumption:

```
// Success response
{
"status": "success",
"data": { ... }, // payload
"meta": { // for paginated lists
"page": 1,
"limit": 20,
"total": 143,
"totalPages": 8
}
}

// Error response
{
"status": "fail", // "fail" for 4xx, "error" for 5xx
"message": "Validation failed: { email: ['Invalid email format'] }"
}
```

Backend Module Summary

Complete API surface — all endpoints, roles, and modules at a glance

Module	# Endpoints	Key Tech / Concerns
Auth (Screen 1)	9	JWT dual-token, bcrypt, Redis blacklist, password reset email
Dashboard (Screen 2)	6	Parallel Prisma aggregates, role-filtered KPIs
Vendors (Screen 3)	8	CRUD, GST validation, S3 logo upload, status machine
RFQ (Screen 4)	11	State machine, Multer/S3 attachments, vendor invitation emails
Quotations (Screen 5)	6	Server-side total calc, versioning, deadline enforcement
Comparison (Screen 6)	1	Normalized scoring, lowest-price detection, matrix query
Approvals (Screen 7)	6	Transactional approve→PO creation, escalation, notifications
PO & Invoices (Screen 8)	9	Auto-number (Redis counter), PDFKit PDF, email dispatch, tax calc
Activity Logs (Screen 9)	6	Audit service, Socket.io real-time push, notification CRUD
Reports (Screen 10)	8	Raw SQL aggregates, XLSX/CSV/PDF export, vendor scoring
Users / Admin (Screen 11)	10	Role assignment, suspension (token revocation), invite flow
Health / System	1	DB + Redis ping, deployment readiness check
TOTAL	81 endpoints	11 modules, full ERP backend coverage

Implementation Priority Order:

1. Part 1–2 (Stack setup + DB schema) — foundation for everything
2. Part 3 (Auth) — all other parts depend on it
3. Part 5 (Vendors) — seeds data for RFQ module
4. Part 6–7 (RFQ + Quotations) — core procurement workflow
5. Part 8–9 (Comparison + Approvals) — decision and approval layer
6. Part 10 (PO + Invoices) — document generation
7. Part 11–12 (Logs + Reports) — observability and analytics
8. Part 13 (User Mgmt) — admin tooling
9. Part 4 (Dashboard) — aggregate of all above
10. Part 14–15 (Cross-cutting + Deployment) — production hardening